



Informe Tarea 1: Laberinto Saltarín

Alumno: Ignacio Alejandro Padilla Palacios
Profesor encargado: Julio Erasmo Godoy Del Campo

1. Introducción

Esta actividad tiene por objetivo el poner en práctica técnicas de inteligencia artificial con el objetivo de resolver el problema del laberinto saltarín. El objetivo es encontrar el camino más corto desde una celda inicial hasta una celda objetivo en una grilla de números enteros positivos, donde cada número define exactamente cuántas casillas puede saltar un agente en las cuatro direcciones cardinales. Para resolver este problema se implementaron los algoritmos de Búsqueda en Profundidad (DFS) y Búsqueda de Costo Uniforme con 2 criterios diferentes.

2. Implementacion

1. Búsqueda en Profundidad (DFS - Depth First Search)

Funcionalidad

El algoritmo de búsqueda en profundidad (DFS) implementado es una versión recursiva que explora todos los posibles caminos desde la celda inicial hasta la celda objetivo, buscando de manera exhaustiva a través del laberinto. El algoritmo sigue explorando un camino hasta llegar a un callejón sin salida o hasta alcanzar la celda de destino. Si se agotan todas las opciones, se retrocede para intentar otras rutas. De esta forma, el algoritmo intenta encontrar una solución a través de la profundización antes de probar otras rutas superficiales.

Estructura

- **get_children:** Esta función se encarga de calcular los hijos válidos de una posición dada en el laberinto, teniendo en cuenta el salto permitido por el valor de la celda actual. Solo se consideran las celdas que no han sido visitadas previamente y que están dentro de los límites del laberinto, garantizando así que no se salgan del mapa y se eviten bucles.
- **dfs_recursive:** Esta función recursiva es el núcleo del algoritmo, que va explorando el laberinto paso a paso. A medida que avanza, va almacenando el camino completo recorrido en una lista llamada `full_path`. Cuando alcanza la celda objetivo, construye el camino válido (`valid_path`) de regreso a la celda inicial, utilizando la pila de recursión para reconstruir el camino desde la meta hasta el origen.
- **dfs:** Es la función principal que se encarga de procesar los datos de entrada, iniciar la llamada recursiva y retornar el laberinto procesado. Al final, devuelve tanto el camino completo recorrido como la solución obtenida, es decir, el camino válido que lleva desde la celda inicial hasta la celda objetivo.

2. Búsqueda de Costo Uniforme (UCS - Uniform Cost Search)

Se implementaron dos versiones del algoritmo UCS, donde se realizó una diferenciación por el criterio de acumulación de costos aplicado dentro del algoritmo.

2.1 UCS v1: Costo Real del Movimiento

Funcionalidad

Esta versión del algoritmo de búsqueda con costo uniforme (UCS v1) considera que el valor de la celda en el laberinto determina el costo de movimiento. Esto implica que desplazarse a una celda con un valor mayor conlleva un mayor costo acumulado. Así, el algoritmo busca minimizar el costo total desde el punto de inicio hasta el objetivo, teniendo en cuenta el valor de cada celda visitada.

Estructura

El algoritmo emplea una estructura de cola de prioridad simulada utilizando `heap.sort()` para asegurar que siempre se explore el nodo con el menor costo acumulado.

- **Diccionario accumulated:** Lleva un registro del menor costo conocido para llegar a cada nodo, lo que permite determinar la ruta más eficiente en términos de costo.
- **Diccionario record:** Guarda los nodos padres de cada celda explorada para reconstruir la ruta final desde el objetivo hacia el inicio, una vez alcanzado el destino.

2.2 UCS v2: Costo Uniforme

Funcionalidad

En esta versión del algoritmo (UCS v2), todas las transiciones entre nodos tienen un costo uniforme de 1, sin considerar el valor de la celda en el laberinto. Esto transforma el problema en un grafo no ponderado, en el cual el objetivo es encontrar la ruta más corta en términos de la cantidad de movimientos necesarios, independientemente de los valores de las celdas.

Estructura

La lógica de UCS v2 reutiliza la estructura de UCS v1, pero con la diferencia de que el costo acumulado se incrementa en 1 en cada paso, sin importar el valor de la celda de destino. Al igual que en UCS v1, se utiliza una cola de prioridad para explorar primero los nodos con el menor costo acumulado y se emplean diccionarios para rastrear el costo y los nodos padres a medida que se recorre el laberinto.

2. Interfaz

Para el desarrollo de la interfaz se utilizaron componentes previamente creados como parte de un proyecto de software, lo que permitió construir una solución visual simple pero funcional. Esta interfaz facilita la visualización del proceso de resolución del *laberinto saltarín* y entrega información detallada sobre su ejecución.

El diseño se divide en dos sectores principales:

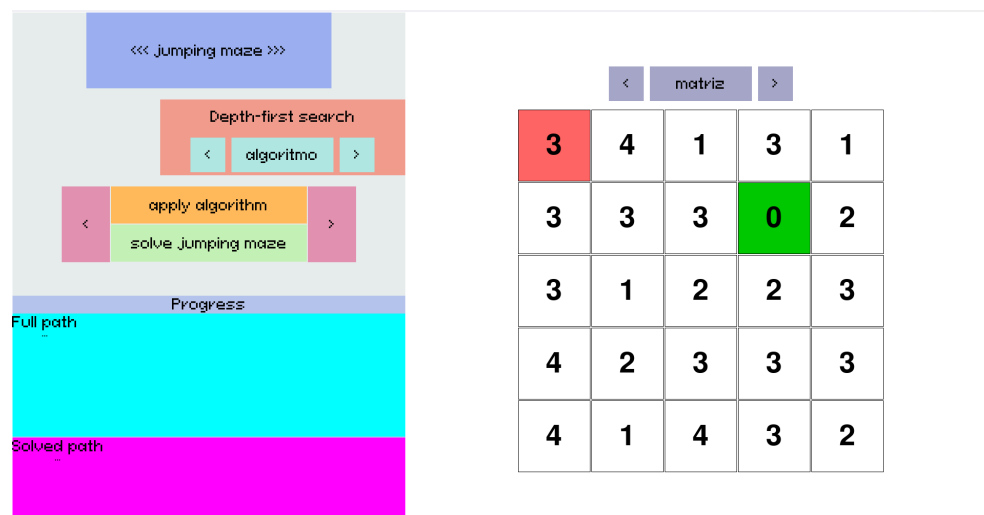
- **Sector izquierdo:**

Aquí se encuentran los **controladores de interacción**. En la parte superior, el usuario puede seleccionar el algoritmo a utilizar para resolver el laberinto. En la parte inferior se ubican dos botones: uno que ejecuta el algoritmo completo y otro que muestra únicamente la solución final obtenida. Justo debajo de estos botones se presenta un área que muestra las **coordenadas de la celda seleccionada**, brindando una retroalimentación instantánea sobre las acciones realizadas.

- **Sector derecho:**

Este sector está dedicado a la **visualización del laberinto**. Incluye un componente que permite seleccionar entre los diferentes laberintos ingresados como parte del input de la tarea, facilitando la navegación entre ellos. Cada laberinto puede visualizarse con claridad, y la solución puede reproducirse paso a paso o mostrarse directamente según la opción seleccionada.

La interfaz, en conjunto, permite al usuario experimentar de forma intuitiva con los distintos algoritmos implementados y comprender su comportamiento al enfrentarse a distintos escenarios.



2. Input y Output de ejemplo

A pesar de que la visualización del desarrollo del algoritmo solo se puede apreciar de manera progresiva a través de la animación generada por el código, se proporcionarán una serie de entradas (inputs) junto con sus respectivos desarrollos mostrados en la interfaz en un momento determinado. Esto permitirá al usuario observar de forma más clara y detallada cómo se resuelven los laberintos en distintos casos.

22	6	6	0	0	5	5
23	5	2	1	1	1	2
24	4	3	4	3	1	5
25	5	2	3	1	4	1
26	3	1	2	1	4	1
27	3	4	1	1	4	5
28	4	3	4	5	2	0

<<< jumping maze >>>

Depth-first search

< algoritmo >

< apply algorithm >

solve jumping maze

Progress

Full path
[(0, 0), (5, 0), (1, 0), (1, 4), (0, 4), (0, 3), (1, 3), (4, 3), (3, 3), (2, 3), (2, 2), (5, 2), (1, 2), (2, 5), (1, 5), (3, 5), (4, 5), (5, 5), (5, 4), (5, 3), (2, 4), (2, 5), (2, 4), (3, 2), (3, 4), (3, 3), (4, 2), (4, 1), (0, 1), (2, 1), (4, 4), (0, 2), (0, 5), (2, 4), (1, 3), (1, 5), (5, 4), (0, 5)]

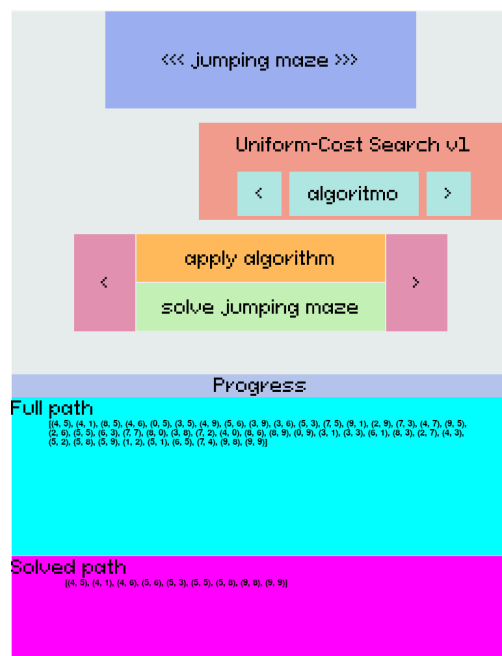
Solved path
[]

<	matriz				>
5	2	1	1	1	2
4	3	4	3	1	5
5	2	3	1	4	1
3	1	2	1	4	1
3	4	1	1	4	5
4	3	4	5	2	0

```

40      20 20 3 3 18 14
41      16 9 1 8 7 5 10 19 13 16 7 3 5 7 5 14 5 14 14 1
42      2 16 1 2 1 1 1 12 11 16 10 4 11 6 16 16 8 1 9 15
43      6 3 12 1 3 11 9 2 12 1 9 10 15 3 5 4 14 2 13 6
44      8 11 5 1 11 16 1 7 18 14 9 16 3 5 6 7 8 1 19 19
45      19 2 4 13 10 19 10 10 14 10 3 16 13 12 5 17 17 12 4 3
46      1 1 12 19 10 4 7 16 10 3 13 1 6 11 7 18 1 15 19 13
47      7 16 5 6 11 9 4 5 7 18 15 4 14 2 1 3 19 17 4 18
48      9 1 7 6 1 9 6 2 8 18 5 2 10 7 9 15 5 17 18 10
49      17 10 7 12 6 12 19 8 13 10 5 15 18 1 11 12 12 7 2 13
50      4 2 13 11 18 9 4 1 19 6 1 15 4 1 6 5 9 12 9 3
51      10 19 1 16 14 16 5 18 18 2 1 18 15 2 2 6 19 14 16 19
52      13 10 4 6 8 13 6 1 2 19 17 16 9 18 9 17 1 6 19 7
53      6 8 16 2 18 12 1 16 13 11 11 12 2 8 7 7 14 9 1 4
54      14 7 15 10 17 4 1 15 10 6 17 7 11 11 4 11 5 9 5 11
55      6 11 18 8 5 1 12 11 18 7 16 17 1 3 2 14 1 4 14 16
56      1 3 15 4 16 17 19 15 7 10 6 14 7 8 14 18 1 18 2 5
57      4 11 17 19 14 18 5 14 19 15 18 13 8 14 8 16 15 11 7 10
58      16 17 11 4 19 9 7 14 10 3 19 8 18 4 1 3 2 5 18 14
59      15 17 4 18 3 10 9 7 3 2 18 10 18 12 0 15 5 6 18 4
60      16 18 18 5 5 14 11 6 2 11 3 17 19 3 1 7 16 1 19 1

```



<	matriz								>
6	2	4	6	6	8	3	7	2	5
3	7	1	6	2	6	8	8	8	5
8	8	7	4	3	6	5	6	5	2
1	8	1	6	1	4	5	7	5	1
7	5	6	4	5	4	1	7	4	9
3	8	4	2	3	3	3	6	4	9
8	3	1	2	3	5	1	4	5	4
5	8	3	1	7	2	7	3	5	9
9	1	4	5	7	5	7	6	8	2
4	6	3	7	6	2	5	9	1	0